Event-Driven Architecture: Patterns, Event Mesh, CQRS and Event Sourcing

Oct 13, 2026 · @Soumyabrata Manna

1. Introduction

Event-driven architecture (EDA) is a style in which systems communicate by publishing events, facts that something happened, and by reacting to the events they care about, instead of calling each other directly. The publisher does not know who listens, so services can be added, changed and scaled independently.

This guide covers the foundations, then the patterns that grow out of them: event mesh, CQRS, event sourcing, the transactional outbox, and choreography versus orchestration. Each section gives the problem, a diagram, a use case, sample code and trade-offs. The code is short and illustrative, and leaves out authentication, configuration and most error handling.

The request for "event segregation" is read here as CQRS (command query responsibility segregation), the pattern that separates writes from reads. It is often paired with event sourcing, so both are covered. Tell me if you meant something else.

Part	Topics	Core question
2. Core concepts	Events, producers, brokers, consumers, delivery guarantees	What are the moving parts?
3. Topologies	Broker and mediator topologies	Who coordinates the flow?
4. Event mesh	Interconnected brokers across regions and clouds	How do events cross boundaries?
5. CQRS	Separate write and read models	How do reads and writes scale apart?
6. Event sourcing	State as a log of events	Can we rebuild state from history?
7. Reliable publishing	Outbox, idempotency, ordering	How do events stay correct?

Part	Topics	Core question
8. Choreography and orchestration	Decentralized or central coordination	Who drives a multi-step process?
9. Event design	Envelope, contracts, versioning	How do events evolve safely?
10-12	Operations, Boomi mapping, selection guide	How do we run and choose?

EDA fits when:

- several consumers need to react to the same fact
- teams must deploy independently
- traffic is bursty or work can be done later
- you need an audit trail or the ability to replay history

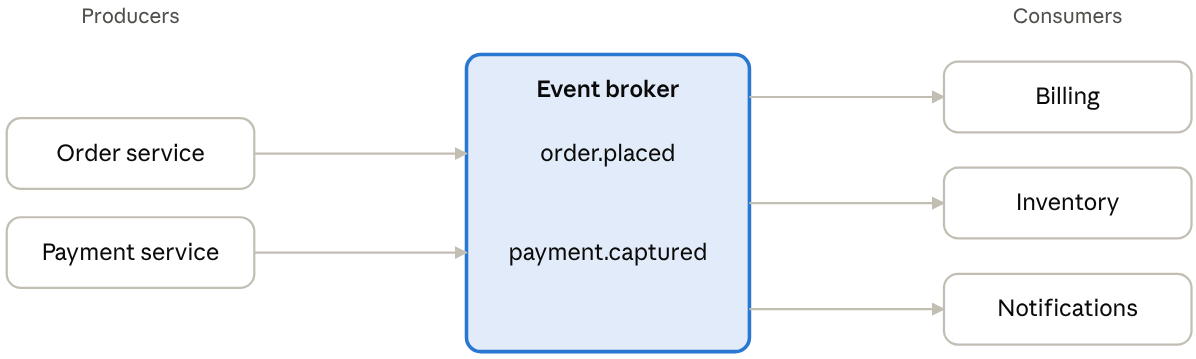
EDA is a poor fit when:

- the work is simple CRUD in one application
- the caller needs an immediate, strongly consistent answer
- the team is small and the number of services is low
- a business rule needs all-or-nothing updates across several records

2. Core Concepts

An event-driven system has three roles: producers announce what happened, a broker carries and stores the events, and consumers react to them.

Producers publish facts; consumers react without the producer knowing them



Producers and consumers never call each other directly

EDA overview · producers, broker, consumers

Adding a fourth consumer needs no change to either producer, which is the main source of the style's flexibility.

2.1 Events, commands and queries

Message	Meaning	Example	Addressed to
Event	A fact that already happened, named in the past tense	`OrderPlaced`	Anyone interested
Command	A request to do something	`PlaceOrder`	One specific handler
Query	A request for data	`GetOrder`	One specific responder

2.2 Kinds of events

Kind	What it carries	Consumer behavior	Trade-off
Event notification	Little more than an id and a type	Calls the source to fetch details	Small events, but extra calls and coupling to the source

Kind	What it carries	Consumer behavior	Trade-off
Event-carried state transfer	The data consumers need	Updates its own copy, no callback	Self-contained, but larger events and duplicated data
Domain event	A business fact such as `InvoicePaid`	Applies business rules	Expresses intent, needs careful naming
Change event	A row-level insert, update or delete (CDC)	Mirrors data changes	Exposes the table structure to consumers

2.3 Delivery guarantees

Guarantee	Meaning	Risk	Typical use
At-most-once	Delivered zero or one times	Loss	Non-critical telemetry
At-least-once	Delivered one or more times	Duplicates	Most business events, with idempotent consumers
Exactly-once	The effect is applied once	Limited to one system and needs support from the platform	Counters and financial totals inside a stream processor

In practice, exactly-once behavior is built from at-least-once delivery plus idempotent processing (section 7.2).

2.4 A minimal event bus

This in-memory sketch shows the idea. A real broker adds durability, delivery guarantees and scaling.

```
from collections import defaultdict

class EventBus:
    def __init__(self):
        self.handlers = defaultdict(list)

    def subscribe(self, event_type, handler):
        self.handlers[event_type].append(handler)

    def publish(self, event_type, payload):
        for handler in self.handlers[event_type]:
            handler(payload)

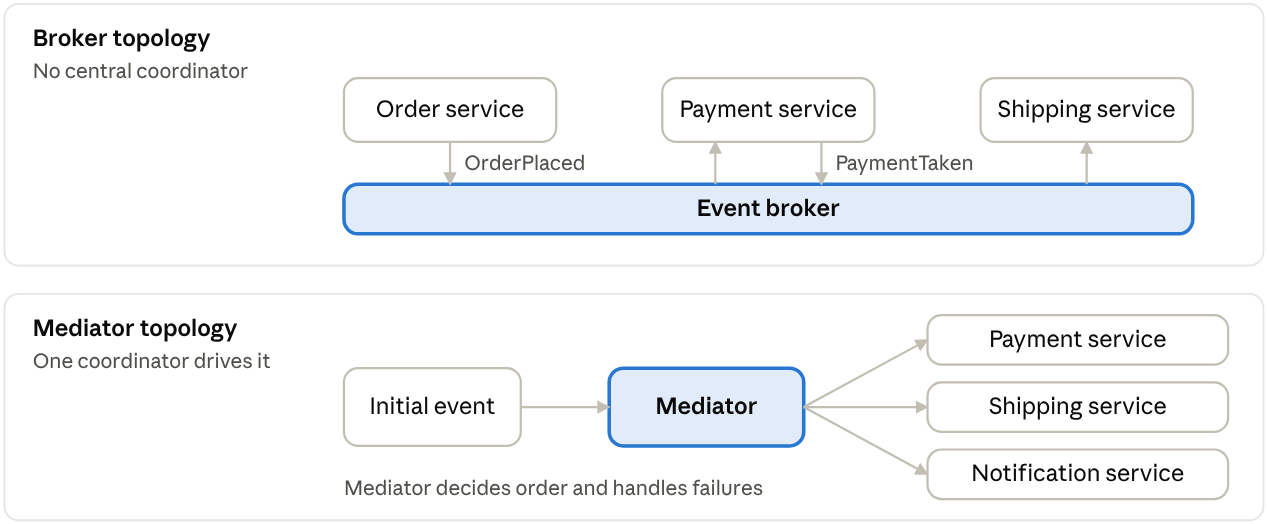
bus = EventBus()
bus.subscribe("OrderPlaced", lambda e: print("billing: invoice",
e["orderId"]))
bus.subscribe("OrderPlaced", lambda e: print("inventory: reserve",
e["orderId"]))
bus.publish("OrderPlaced", {"orderId": "A-1001", "total": 59.9})
```

Trade-offs: loose coupling and easy fan-out. Flow is harder to follow than a call stack, and consistency becomes eventual, so design for it from the start.

3. Topologies

Event-driven systems take one of two basic shapes, depending on whether anyone coordinates the flow.

A broker topology has no coordinator; a mediator topology has one



broker vs mediator topology · 2 flows

In a broker topology, services chain themselves through events. In a mediator topology, a central component tells each service what to do next.

3.1 Broker topology

Each service reacts to an event and may publish a follow-up event. No component owns the whole flow. **Use case:** `OrderPlaced` triggers payment, whose `PaymentTaken` event triggers shipping, with no central owner. The sample is the payment service, using `confluent-kafka`.

```
while True:
    msg = consumer.poll(1.0)  # subscribed to "order.placed"
    if msg is None or msg.error():
        continue
    order = json.loads(msg.value())
    receipt_id = charge_card(order["customerId"], order["total"])
    producer.produce(
        "payment.taken",
        json.dumps({"orderId": order["orderId"], "receiptId": receipt_id}),
    )
    producer.flush()
```

Trade-offs: very loose coupling and easy scaling. The overall flow lives nowhere in one place, so it is hard to see, and failures that need coordinated recovery are hard to handle.

3.2 Mediator topology

A mediator receives the initial event, then sends commands to processors in a defined order, tracks progress and handles errors. **Use case:** a loan application that must pass fixed approval steps.

```
def run_order_workflow(order: dict) -> None:
    steps = ["payment.charge", "inventory.reserve", "shipping.create"]
    for command in steps:
        result = send_command(command, order, timeout=30)  # request/reply
over the broker
        if not result["ok"]:
            publish_event("OrderFailed", {"orderId": order["orderId"],
"failedAt": command})
            return
    publish_event("OrderCompleted", {"orderId": order["orderId"]})
```

Trade-offs: the flow is visible and error handling is central. The mediator becomes a coupling point and must be scaled and made highly available. Section 8 compares this with choreography in more depth.

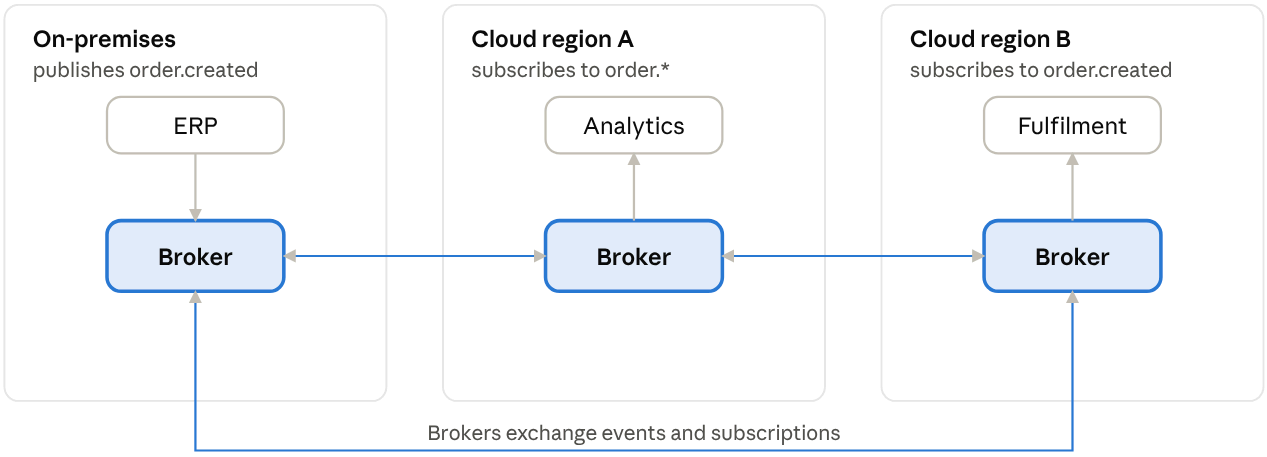
3.3 Choosing between them

Criterion	Broker topology	Mediator topology
Coordination	None; services react	One component directs
Visibility of the flow	Low, needs tracing	High
Error handling	Spread across services	Central
Coupling	Lowest	Services depend on the mediator's commands
Best for	Simple, independent reactions	Multi-step processes with rules and recovery

4. Event Mesh

An event mesh is a network of interconnected event brokers that lets an event published in one environment reach subscribers in any other, so applications need not know where their counterparts run. The term is used by analysts and by vendors such as Solace. Kafka-based stacks reach a similar result by replicating topics between clusters.

An event mesh links brokers so an event reaches subscribers in any region



event mesh · 3 environments

Publishers and subscribers talk only to their nearest broker. The mesh carries each event to the brokers that have a matching subscriber.

4.1 How it works

- **Topic hierarchy:** topics such as `retail/orders/created/eu` let subscribers use wildcards (`retail/orders/created/+`).
- **Subscription-aware routing:** in many mesh products brokers share their subscriptions, so an event is forwarded only to brokers with interested consumers.
- **Environment neutrality:** brokers run on premises, in several clouds and at the edge, and applications use standard protocols such as MQTT, AMQP or JMS.
- **Delivery per link:** each hop can persist and acknowledge events, so a broken link delays delivery instead of losing events.

4.2 Use case

A retailer keeps its ERP on premises, runs analytics in one cloud region and fulfilment in another. When the ERP publishes `order.created`, analytics and the nearest fulfilment centre receive it without a point-to-point integration between any pair. The same setup helps a gradual cloud migration, because a consumer can move to the cloud without its publisher changing.

4.3 Sample code

The publisher and subscriber connect to different local brokers, and the mesh moves the event between them. The sample uses MQTT with `paho-mqtt` 2.x, which many brokers support.

```
import json
import paho.mqtt.client as mqtt

# Publisher, connected to the on-premises broker
client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
client.connect("broker.onprem.example.com", 1883)
event = {"orderId": "A-1001", "region": "eu", "total": 59.9}
client.publish("retail/orders/created/eu", json.dumps(event), qos=1)
```

```
# Subscriber, connected to the cloud broker
def on_message(client, userdata, msg):
    print(msg.topic, json.loads(msg.payload))

client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
client.on_message = on_message
client.connect("broker.cloud.example.com", 1883)
client.subscribe("retail/orders/created/+", qos=1)  # every region
client.loop_forever()
```

With Kafka, Apache MirrorMaker 2 replicates topics between clusters. This configuration copies order topics from an on-premises cluster to a cloud cluster. By default, replicated topics are prefixed with the source cluster alias, for example `onprem.orders`.

```
clusters = onprem, cloud
onprem.bootstrap.servers = kafka.onprem.example.com:9092
cloud.bootstrap.servers = kafka.cloud.example.com:9092
onprem->cloud.enabled = true
onprem->cloud.topics = orders.*
replication.factor = 3
```

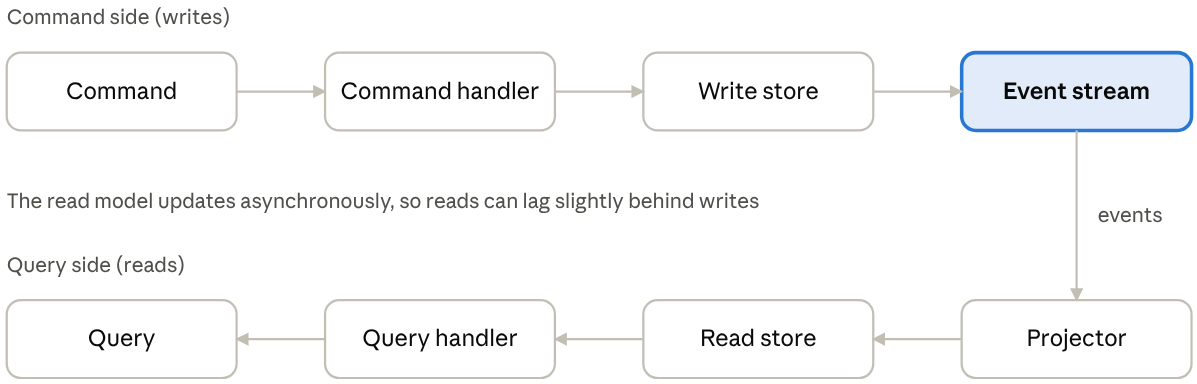
4.4 Trade-offs

- **Gains:** location transparency, regional resilience, and no pairwise integrations between environments.
- **Costs:** more brokers to operate, topic naming and access governance across environments, added latency between regions, and data-residency rules that restrict which events may cross a border.
- **When to start:** a single broker is enough while everything runs in one environment. Add a mesh when events must cross clouds, regions or the edge.

5. CQRS (Command Query Responsibility Segregation)

CQRS separates the model that changes data (commands) from the model that reads it (queries), so each side can be shaped, stored and scaled on its own.

Writes and reads use separate models, joined by an event stream



CQRS · write and read paths

The command side validates and records changes, then publishes events. A projector consumes those events and keeps a read store shaped for the screens and APIs that query it.

5.1 Why separate the two sides

- **Different shapes:** writes favor normalized data and strict rules, while reads favor denormalized views that need no joins.
- **Different scale:** reads usually outnumber writes, so the read store can be replicated or cached without touching the write path.
- **Different stores:** a relational database can take writes while a document store or search index serves reads.
- **Different security:** who may change data is often a smaller group than who may see it.

5.2 Use case

An online shop writes orders to a relational database with strict validation. The customer's order-history page reads from a denormalized document per customer, and product search reads from a search index. Both read models are projections of the same order events, and each can be rebuilt from them.

5.3 Sample code

The `db` and `read_db` helpers are placeholders for your data access code.

```
# Command side: validate, change state, record the event
def place_order(cmd, db, outbox):
    if cmd["total"] <= 0:
        raise ValueError("total must be positive")
    db.insert("orders", {
        "id": cmd["orderId"], "customer_id": cmd["customerId"],
        "total": cmd["total"], "status": "PLACED",
    })
    outbox.append({
        "type": "OrderPlaced", "orderId": cmd["orderId"],
        "customerId": cmd["customerId"], "total": cmd["total"],
    })
```

```
# Projector: keep a denormalized read model up to date (safe to run twice)
def on_order_placed(event, read_db):
    key = event["customerId"]
    summary = read_db.get("customer_orders", key) or {"customerId": key,
"orders": []}
    summary["orders"] = [o for o in summary["orders"] if o["orderId"] !=
event["orderId"]]
    summary["orders"].append(
        {"orderId": event["orderId"], "total": event["total"], "status":
"PLACED"}
    )
    read_db.put("customer_orders", key, summary)

# Query side: one cheap lookup, no joins
def get_customer_orders(customer_id, read_db):
    return read_db.get("customer_orders", customer_id)
```

The `outbox` list stands for the transactional outbox in section 7.1, which makes the state change and the event succeed or fail together.

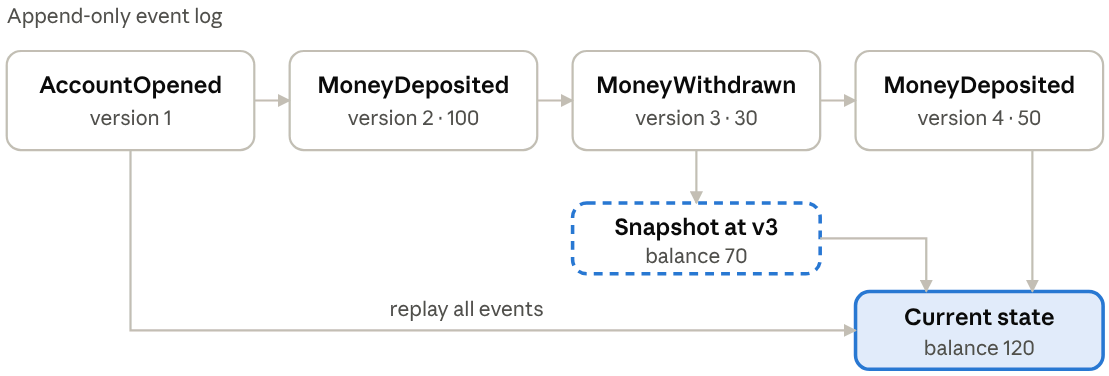
5.4 Trade-offs

- **Gains:** read and write models that fit their jobs, independent scaling, and read models you can rebuild or add without touching the write side.
- **Costs:** eventual consistency, more components to run, and duplicated data. After a write, a user may briefly see old data. Common fixes are returning the written data in the command response or showing a pending state until the projection catches up.
- **Scope it:** apply CQRS to the parts of a system where read and write needs really differ, not everywhere. Simple CRUD areas do not need it.

6. Event Sourcing

Event sourcing stores every change to an entity as an immutable event and derives the current state by replaying those events, instead of storing only the latest state.

State is the sum of past events; a snapshot shortens the replay



A full replay always works; a snapshot plus newer events only saves time

event sourcing · log, replay, snapshot

The log is the source of truth. Current state is a fold over the events, and a snapshot saves replaying from the very start.

6.1 How it works

- **Append-only log:** events are never updated or deleted. A correction is a new event.
- **State by replay:** applying events in order rebuilds the state at any point in time.
- **Snapshots:** a saved state at a known version, so replay needs only the later events.
- **Projections:** read models built from the log, which can be dropped and rebuilt (the read side of CQRS).
- **Optimistic concurrency:** each append states the version it expects, and a conflict is rejected.

6.2 Use case

A bank ledger must show every transaction, answer "what was the balance on March 3?" and let a fixed bug be corrected by replaying history. Order lifecycles, claims processing and inventory movements fit the same way.

6.3 Sample code

The account state is rebuilt by folding events:

```
from dataclasses import dataclass

@dataclass
class Account:
    balance: int = 0
    version: int = 0

def apply(state: Account, event: dict) -> Account:
    kind = event["type"]
    if kind == "MoneyDeposited":
        return Account(state.balance + event["amount"], state.version + 1)
    if kind == "MoneyWithdrawn":
        return Account(state.balance - event["amount"], state.version + 1)
    return Account(state.balance, state.version + 1)  # AccountOpened

def replay(events: list[dict], snapshot: Account | None = None) -> Account:
    state = snapshot or Account()
    for event in events:  # with a snapshot, pass only the events after it
        state = apply(state, event)
    return state
```

A command checks the rules against the replayed state, then appends with the version it expects:

```
def withdraw(store, account_id: str, amount: int) -> None:
    state = replay(store.load(account_id))
    if state.balance < amount:
        raise ValueError("insufficient funds")
    store.append(
        account_id,
        expected_version=state.version,
        event={"type": "MoneyWithdrawn", "amount": amount},
    )
```

In a relational database the primary key enforces the version check (PostgreSQL syntax):

```
CREATE TABLE events (
    stream_id   text        NOT NULL,
    version     integer     NOT NULL,
    type        text        NOT NULL,
    data        jsonb       NOT NULL,
    created_at  timestamptz NOT NULL DEFAULT now(),
    PRIMARY KEY (stream_id, version)
);

-- Fails if another writer already used version 5 for this stream
INSERT INTO events (stream_id, version, type, data)
VALUES ('account-42', 5, 'MoneyWithdrawn', '{"amount": 30}');
```

6.4 Trade-offs

- **Gains:** a complete audit trail, queries about past states, easy debugging by replay, and read models you can rebuild.
- **Costs:** a different way of thinking, event schemas that must evolve carefully (section 9.4), and no ad hoc queries on the log itself, so you need projections.
- **Personal data:** an immutable log conflicts with deletion requests. Keep personal data out of events, or encrypt it per person and discard the key.
- **Relationship to CQRS:** they pair well but are independent. You can use CQRS without event sourcing, and event sourcing without separate read stores for simple cases.

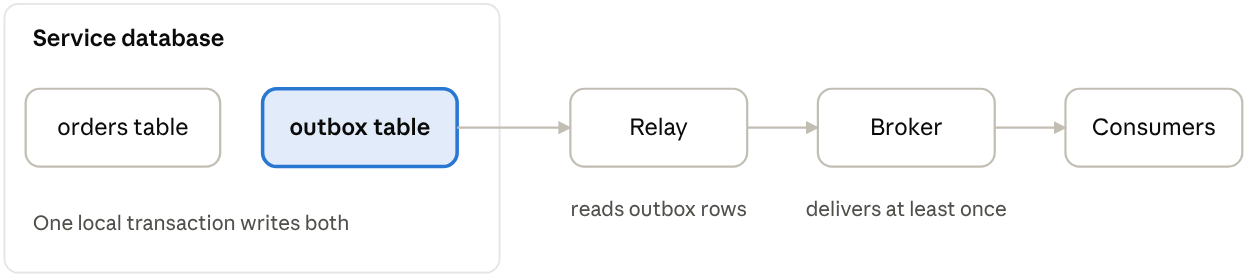
7. Reliable Publishing, Idempotency and Ordering

An event-driven system is correct only if every state change produces its event, no event causes a side effect twice, and events that must stay in order do.

7.1 Transactional outbox

The problem is the dual write: if a service saves data and then publishes an event, a crash between the two leaves them out of step. The outbox stores the event in the same database transaction as the data, and a relay publishes it afterwards.

The outbox makes the data change and the event succeed or fail together



transactional outbox · 4 steps

Use case: an order is saved and `OrderPlaced` must be published, with neither ever happening alone.

```
BEGIN;
INSERT INTO orders (id, customer_id, total, status)
VALUES ('A-1001', 'C-7', 59.90, 'PLACED');

INSERT INTO outbox (id, aggregate_id, type, payload)
VALUES (gen_random_uuid(), 'A-1001', 'OrderPlaced',
        '{"orderId": "A-1001", "total": 59.90}');
COMMIT;
```

A relay then publishes unsent rows (PostgreSQL and `psycopg2` shown):

```
import json

def relay_outbox(conn, producer):
    with conn:
        cur = conn.cursor()
        cur.execute(
            "SELECT id, aggregate_id, type, payload FROM outbox "
            "WHERE published_at IS NULL ORDER BY created_at "
            "LIMIT 100 FOR UPDATE SKIP LOCKED"
        )
        for row_id, aggregate_id, event_type, payload in cur.fetchall():
            producer.produce(
                "orders.events",
                key=aggregate_id,
                value=json.dumps(payload),
                headers={"event-type": event_type, "event-id": str(row_id)},
            )
            cur.execute("UPDATE outbox SET published_at = now() WHERE id =
%s", (row_id,))
        producer.flush()
```

If the relay crashes after publishing but before the commit, the row is sent again. That is why the outbox gives at-least-once delivery and consumers must be idempotent. Instead of polling, a CDC tool such as Debezium can read the outbox table from the database log.

Trade-offs: no lost events and no distributed transaction. It adds a table and a relay to operate, and a small publishing delay.

7.2 Idempotent consumers

A consumer is idempotent when handling the same event twice has the same effect as handling it once. Some operations are naturally safe, such as setting a value or an upsert. Others, such as adding to a balance, need protection. The sample records each event id in the same transaction as the change (PostgreSQL syntax).

```
def handle_event(conn, event_id: str, apply_change) -> None:
    with conn:  # one transaction
        cur = conn.cursor()
        cur.execute(
            "INSERT INTO processed_events (event_id) VALUES (%s) ON CONFLICT
DO NOTHING",
            (event_id,),
        )
        if cur.rowcount == 0:
            return  # duplicate: already handled
        apply_change(cur)
```

7.3 Ordering

Brokers keep order only within one queue or partition, not across all of them. Give every event a key such as the aggregate id, so all events for one entity land in the same partition and stay in order. For extra safety, include a version number and ignore events that are older than what you hold.

```
producer.produce("orders.events", key=order_id, value=payload)  # same key,
same partition
```

```
def apply_if_newer(read_db, event):
    current = read_db.get("orders", event["orderId"]) or {"version": 0}
    if event["version"] <= current["version"]:
        return  # old or duplicate event: ignore
    # A gap (for example version 5 arriving after 3) can be parked until 4
arrives.
    read_db.put("orders", event["orderId"], {**event["data"], "version":
event["version"]})
```

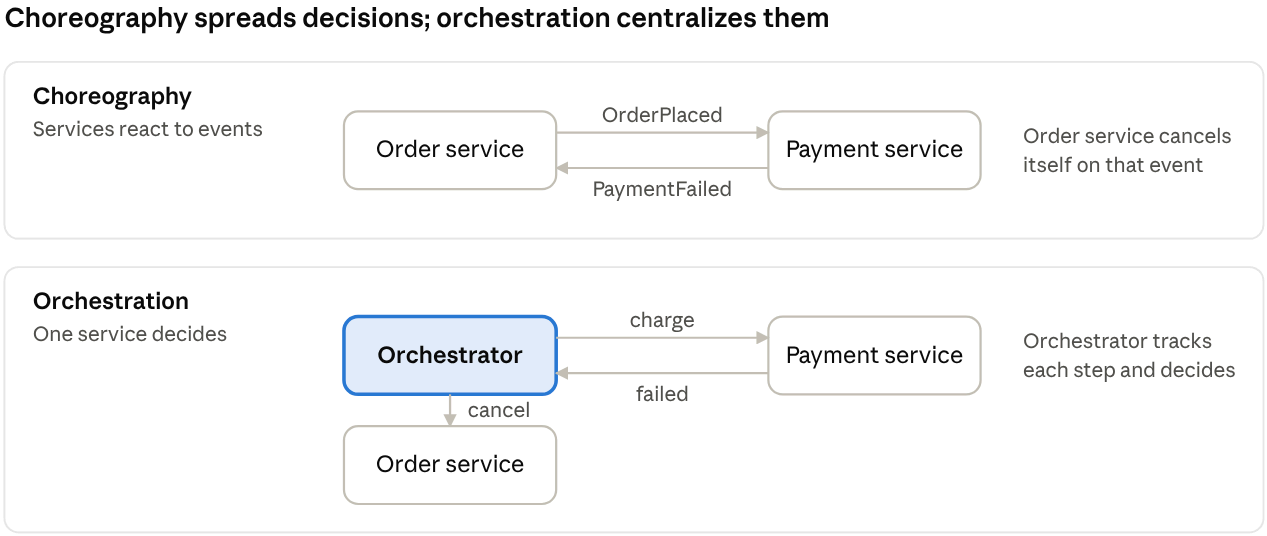
7.4 Failure and remedy

Failure	Remedy
Event lost after the database commit	Transactional outbox
Duplicate delivery	Idempotent consumer with an event id
Events processed out of order	Partition key plus version checks

Failure	Remedy
A message that always fails	Bounded retries, then a dead-letter queue

8. Choreography vs Orchestration

Both approaches coordinate a process that spans services. They differ in who decides the next step.



choreography vs orchestration · payment failure

The diagram follows one failure. In choreography the payment service announces `PaymentFailed` and the order service decides to cancel. In orchestration the orchestrator sees the failure and tells the order service to cancel.

8.1 Choreography

Each service reacts to events and publishes its own. No component holds the whole process. **Use case:** order, payment and shipping services that each react to the previous service's event.

```
# Order service: reacts to the payment service's events
def on_event(event: dict) -> None:
    if event["type"] == "PaymentFailed":
        cancel_order(event["orderId"], reason=event["reason"])
        publish("OrderCancelled", {"orderId": event["orderId"]})
```

Trade-offs: minimal coupling and no central bottleneck. The process is implicit, so it is hard to see, change or monitor end to end, and cyclic dependencies can creep in.

8.2 Orchestration

An orchestrator keeps the process state, sends commands to services and reacts to their replies, including running compensations on failure. **Use case:** the same order process, with one place that shows every step and its status.

```
STEPS = ["charge_payment", "reserve_stock", "create_shipment"]
COMPENSATIONS = {"charge_payment": "refund_payment", "reserve_stock":
"release_stock"}

def on_reply(state: dict, reply: dict) -> None:
    order = {"orderId": state["orderId"]}
    if reply["ok"]:
        state["completed"].append(reply["step"])
        next_index = len(state["completed"])
        if next_index == len(STEPS):
            publish("OrderCompleted", order)
        else:
            send_command(STEPS[next_index], order)
    else:
        for step in reversed(state["completed"]):
            if step in COMPENSATIONS:
                send_command(COMPENSATIONS[step], order)
        publish("OrderFailed", {**order, "failedAt": reply["step"]})
    save_state(state)  # persist so the process survives a restart
```

Workflow engines such as Temporal, Camunda or AWS Step Functions provide this state handling, retries and timers, so you rarely need to build it yourself.

Trade-offs: the flow is explicit and easy to monitor, and failure handling lives in one place. The orchestrator is a dependency that must be highly available, and it can attract business logic that belongs in the services.

8.3 Comparison

Criterion	Choreography	Orchestration
Who decides the next step	Each service, from events	The orchestrator

Criterion	Choreography	Orchestration
Visibility of the process	Low, needs distributed tracing	High, state is in one place
Coupling	Services know event contracts	Services know the orchestrator's commands
Changing the process	Touches several services	Mostly one place
Failure handling	Spread across services	Central, with compensations
Best for	Short, simple reactions across teams	Long, multi-step processes with rules

Many systems combine them: choreography between business domains, and orchestration for a complex process inside one domain.

9. Event Design, Contracts and Versioning

Events are a public contract between teams, so their names, fields and change rules deserve as much care as an API.

9.1 Design guidelines

- **Name events as past-tense business facts:** `OrderPlaced`, not `OrderUpdated` or `SaveOrder`.
- **One fact per event:** a consumer should not need to guess which of several changes happened.
- **Carry identity and context:** a unique id, a source, a type with a version, a timestamp, and a correlation id that links related events.
- **Do not leak internals:** publish business concepts, not your table structure.
- **Choose thin or fat on purpose:** thin events carry ids and force a callback, fat events carry the data consumers need (section 2.2).

9.2 A standard envelope: CloudEvents

CloudEvents is a vendor-neutral specification for the metadata around an event. Its required attributes are `id`, `source`, `specversion` and `type`. A JSON example:

```
{
  "specversion": "1.0",
  "id": "5f1e2c0a-8f1b-4b6a-9a52-1f3e2d4c5b6a",
  "source": "/shop/orders",
  "type": "com.example.order.placed.v1",
  "subject": "A-1001",
  "time": "2026-10-13T14:05:00Z",
  "datacontenttype": "application/json",
  "data": {
    "orderId": "A-1001",
    "customerId": "C-7",
    "total": 59.90,
    "currency": "USD"
  }
}
```

9.3 Documenting the contract: AsyncAPI

AsyncAPI describes channels, messages and who sends or receives them, much as OpenAPI does for REST. This AsyncAPI 3.0 fragment documents the order event:

```
asyncapi: 3.0.0
info:
  title: Order events
  version: 1.0.0
channels:
  orderPlaced:
    address: orders.placed
    messages:
      OrderPlaced:
        $ref: '#/components/messages/OrderPlaced'
operations:
  publishOrderPlaced:
    action: send
    channel:
      $ref: '#/channels/orderPlaced'
components:
  messages:
    OrderPlaced:
      payload:
        type: object
```

```
        required: [orderId, total]
        properties:
          orderId: { type: string }
          total: { type: number }
```

9.4 Schema evolution and versioning

Producers and consumers deploy at different times, so a schema change must not break the other side.

Change	Effect on existing consumers
Add an optional field with a default	Safe
Add a required field	Breaking
Remove or rename a field	Breaking
Change a field's type	Breaking
Change a field's meaning	Breaking, and hard to detect

For a breaking change, publish a new event type (`order.placed.v2`) alongside the old one and retire the old one after consumers migrate. An Avro schema shows a safe change: the new `currency` field has a default, so readers of old events still work.

```
{
  "type": "record",
  "name": "OrderPlaced",
  "namespace": "com.example.orders",
  "fields": [
    {"name": "orderId", "type": "string"},
    {"name": "total", "type": "double"},
    {"name": "currency", "type": "string", "default": "USD"}
  ]
}
```

Consumers should be tolerant readers: use the fields they need, ignore unknown ones and supply defaults for missing ones. When old events live forever in an event store, an upcaster converts them to the current shape on read.

```
def parse_order_placed(data: dict) -> dict:
    return {
        "orderId": data["orderId"],
        "total": data["total"],
        "currency": data.get("currency", "USD"),  # default for events from
before the field existed
    }

def upcast(event: dict) -> dict:
    if event["type"] == "order.placed.v1":
        return {**event, "type": "order.placed.v2",
                "data": {**event["data"], "channel": "web"}}
    return event
```

9.5 Schema registry and compatibility checks

A schema registry (for example Confluent Schema Registry, Apicurio or AWS Glue Schema Registry) stores schemas and rejects incompatible changes. Run the compatibility check in the build pipeline so a breaking change fails before it reaches production.

10. Operations: Observability, Testing, Governance and Anti-Patterns

Because no single call stack shows what happened, an event-driven system needs deliberate tooling to be understandable and safe to change.

10.1 Observability

Propagate a trace context in event headers so one business transaction can be followed across services. The sample uses OpenTelemetry and Kafka headers.

```
import json
from opentelemetry import propagate, trace

tracer = trace.get_tracer(__name__)

def publish(producer, topic, event):
    headers = {}
    propagate.inject(headers)  # adds the W3C traceparent header
    producer.produce(topic, value=json.dumps(event), headers=headers)

def consume(msg, handler):
    carrier = {k: v.decode() for k, v in (msg.headers() or [])}
    ctx = propagate.extract(carrier)
    with tracer.start_as_current_span("handle-event", context=ctx):
        handler(json.loads(msg.value()))
```

Signal	Why it matters	Alert when
Consumer lag	Shows whether consumers keep up	It keeps rising over several intervals
Dead-letter queue depth	Counts messages that failed for good	It is above zero and not draining
End-to-end latency	Measures time from publish to final effect	It exceeds the business target
Publish failure rate	Reveals broker or relay problems	It rises above the normal baseline
Schema check failures	Catches incompatible changes	Any failure in the pipeline

10.2 Testing

- **Unit tests** for each handler, with events as plain input data.
- **Contract tests** that check producers and consumers against the same schema, so a change that breaks a consumer fails before release.
- **Failure tests** that deliver duplicates and out-of-order events on purpose.
- **Replay tests** that run a recorded stream through new code and compare the result.

```
def test_duplicate_event_is_applied_once(conn):
    handle_event(conn, "evt-1", add_credit(10))
    handle_event(conn, "evt-1", add_credit(10))  # redelivery of the same
event
    assert balance(conn) == 10
```

10.3 Governance

- **Ownership:** every topic and event type has a named owning team.
- **Naming:** a convention such as `domain.entity.event.version`, applied everywhere.
- **Access control:** producers and consumers authenticate, and each may use only the topics it needs.
- **Retention:** set how long events are kept, per topic, based on replay needs and privacy rules.
- **Catalog:** a searchable list of events and schemas (for example AsyncAPI documents) so teams can find and reuse them.

10.4 Common anti-patterns

- **Events used as remote calls:** a service publishes an event and then blocks waiting for a reply event. Use request/reply instead.
- **Commands disguised as events:** `SendEmail` is an instruction, not a fact. Publish `OrderPlaced` and let the email service decide.
- **CRUD events:** `OrderUpdated` says nothing about what changed or why.
- **A distributed monolith:** services so dependent on each other's events that they must deploy together.
- **The log as a shared database:** several teams read another team's internal events as their source of truth.
- **No dead-letter path or schema governance:** failures stay invisible and changes break consumers silently.
- **Event sourcing and CQRS everywhere:** use them where history or separate read models pay off.

11. Mapping to Boomi

Boomi covers much of this architecture with processes, connectors and runtimes. The table lists the usual building blocks. Component names and features change between releases, so confirm details in the current Boomi documentation, especially the items marked "verify".

Concept	Typical Boomi building blocks
Event broker (topics, queues)	Boomi Event Streams, or the Kafka, JMS, Amazon SQS/SNS, Azure Service Bus and Google Pub/Sub connectors
Producers and consumers	Processes using a connector's send operation (producer) or a listener/subscribe operation (consumer)
Broker topology (choreography)	Independent processes, each subscribed to a topic and publishing follow-up events
Mediator and orchestration	One orchestrating process using Process Call, Decision and Try/Catch shapes; Boomi Flow for steps that need a person
Event mesh	Atoms in each environment connected to their local broker, with mesh behavior from the broker platform or cross-cluster replication (verify)
CQRS	Separate command and query API processes, plus a projector process that updates a read store
Event sourcing	A process that appends events to a database table or event store; there is no dedicated event store shape (verify)
Transactional outbox	Database connector writing the data and the outbox row in one transaction, then a scheduled or listener process publishes the rows
Idempotent consumer	Document cache or database lookup keyed on the event id, tested with a Decision shape
Retry and dead-letter handling	Try/Catch with retries, failed documents written to a queue or table, and notifications

Concept	Typical Boomi building blocks
Event contracts	JSON or XML profiles kept as versioned components; API Management for REST contracts
Observability	Process reporting and execution logs, with a correlation id carried in dynamic process properties

Practical guidance:

- Keep each event-handling process small, with one responsibility.
- Put the cross-cutting parts, such as envelope parsing, duplicate checks, retry and dead-letter handling, into reusable subprocesses in a shared library.
- Version event profiles as separate components, so a breaking change is a new component and not an edit.

12. Selection Guide and Design Checklist

Start with the simplest style that meets the need, and add patterns only for a problem you actually have.

Situation	Pattern	Section
Several services must react to the same fact	Publish/subscribe with a broker	2, 3.1
Simple, independent reactions across teams	Broker topology (choreography)	3.1, 8.1
Multi-step process with rules and recovery	Mediator or orchestration	3.2, 8.2
Events must cross clouds, regions or the edge	Event mesh	4
Reads and writes need different shapes or scale	CQRS	5
Full history, audit or point-in-time state needed	Event sourcing	6
A data change and its event must never diverge	Transactional outbox	7.1
Messages may be delivered more than once	Idempotent consumer	7.2

Situation	Pattern	Section
Events for one entity must stay in order	Partition key plus version checks	7.3
Many teams share events over time	Event contracts, registry, versioning	9
Failures are invisible or hard to trace	Tracing, lag and dead-letter alerts	10

Design checklist

- [ ] Is an event the right fit, or does the caller need an immediate answer?
- [ ] Are events named as past-tense business facts, with an owner?
- [ ] Does every event carry an id, source, type, version, time and correlation id?
- [ ] Is each state change and its event published together (outbox or CDC)?
- [ ] Is every consumer idempotent and tolerant of unknown fields?
- [ ] Is ordering handled with keys and version checks where it matters?
- [ ] Is there a dead-letter path with an alert on its depth?
- [ ] Are schemas registered, versioned and checked for compatibility in the build?
- [ ] Can one business transaction be traced across services?
- [ ] Do consumers cope with read models that lag behind writes?
- [ ] Are retention, access control and personal-data rules defined per topic?